Advanced Terraform on Azure
Lab6b Pipeline Automation

Expected Duration

Part 1 of this lab is expected to take between 10-15 minutes
Part 2 of this lab is expected to take 30-40 minutes
Part 3 of this lab is expected to take 40-50 minutes
Part 4 of this lab is expected to take 30-40 minutes

Total expected lab time is therefore between 110 - 145 minutes

Take breaks between parts as needed.

Prerequisites

This lab assumes the existence of a remote state backend, as created in Lab 4. Run this lab now if you do not have remote state backend configured in your Azure subscription.

This lab also assumes that Lab6a has been completed. If this has **not** been done, **or** you have r**estarted** (not paused/resumed) the lab session since, then run the following commands from a VSC terminal session ...

```
cd c:\qatfadvaz\lab6\original\reset6b
.\reset6b.ps1
```

Update **providers.tf** in **lab6\guided\envs\dev** with your subscription id and the suffix used with your storage account name. Repeat this for **envs/test/providers.tf** and **envs/prod/providers.tf**

You will also need to re-clone a copy of the network and security module repos you created in Lab6a..

```
cd c:\
mkdir module-repos
cd module-repos
git clone {your network repo url}
git clone {your security repo url}
cd c:\module-repos\terraform-azure-module-network
git config --local user.email "{repo-user-email}"
git config --local user.name "{repo-username}"
cd c:\module-repos\terraform-azure-module-security
git config --local user.email "{repo-user-email}"
git config --local user.name "{repo-username}"
```

Re-add your repo folder to VSC Explorer

Part 1. Local PS Pipeline-Enforced Changes

Teaching Points

So far you have promoted module versions manually across DEV, TEST, and PROD. You made explicit decisions about when promotion should occur and represented promotion by changing pinned module versions.

Now we introduce **pipelines** — not to make decisions for you, but to **enforce the rules you already defined**.

Here we demonstrate how platform teams prevent mistakes by automating *execution*, while keeping *decision-making* firmly in human hands.

By the end of these steps , you will understand how to:

- Use pipelines to enforce Terraform promotion rules
- Separate promotion decisions from execution
- Prevent unsafe or out-of-sequence environment changes
- Replace manual Terraform execution with controlled automation
- Prepare for full CI/CD adoption without changing architecture

When people hear the word *pipeline*, they often think of GitHub Actions or Azure DevOps. That is **not** what this part of the lab introduces, that comes in Part 2. Here, a *pipeline* simply means:

A repeatable sequence of commands that a machine runs instead of a human, with rules enforced automatically.

This part of the lab is organised into the following phases:

Phase 0 – Review the Promotion Model
Phase 1 – Review the Local Pipeline
Phase 2 – Pipeline-Applied DEV Deployment
Phase 3 – Enforce Promotion Order
Phase 4 – Promote through the pipeline

Phase 0 – Review the Promotion Model

Before introducing automation, the following rules have been established ....

- Promotion is a module version pin change
- DEV validates new module versions
- TEST cannot advance ahead of DEV
- PROD cannot advance ahead of TEST

Promotion also includes reverting a module version pin

Phase 1 – Review the Local Pipeline

In this phase, you introduce a **local pipeline script**.

This script replaces you typing Terraform commands by hand.

The pipeline will:

- Run Terraform in a fixed order
- Enforce promotion rules
- Refuse to run if rules are violated
- Apply Terraform non-interactively

The pipeline does **not**:

- Decide when to promote
- Change module versions
- Auto-promote environments
- Modify Terraform structure

You have been given a set of PowerShell scripts at **artifacts/pipelines/powershell.** These scripts replace manual terraform init / plan / apply operations.

Phase 2 - Pipeline-Applied Changes 1

A decision has been made to promote network module version **v1.0.0** in all environments.

1. Update the pinned version of the **network** module to **v1.0.0** in **main.tf** of each of the environments
2. Open a new terminal session and run each of these in turn …

   c:\qatfadvaz\artifacts\pipelines\powershell\change-dev.ps1
   c:\qatfadvaz\artifacts\pipelines\powershell\change-test.ps1
   c:\qatfadvaz\artifacts\pipelines\powershell\change-prod.ps1
3. Observe:

   Terraform runs without manual input
   Commands execute in a fixed order
4. In the Portal, verify the VNet manager tag is 'Michael Coulling-Green (NetMod v1.0.0)'

Phase 3 - Pipeline-Applied Changes 2

A decision has been made to promote network module version **v1.0.1** in the DEV environment.

5. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **dev** environments
6. Run …

   ```
   c:\qatfadvaz\artifacts\pipelines\powershell\change-dev.ps1
   ```
7. In the Portal, verify the VNet manager tag updates to "John Robinson (NetMod v1.0.1)' from 'Michael Coulling-Green (NetMod v1.0.0)' in the dev resource group.

Phase 4 – Verify Environment Order Enforcement

In this phase, you observe how the pipeline prevents unsafe promotion by trying to promote v1.0.1 of the network module to PROD, bypassing TEST.

8. Update the pinned version of the **network** module to **v1.0.1** in **main.tf** of the **prod** environments
9. Run …

   ```
   c:\qatfadvaz\artifacts\pipelines\powershell\change-prod.ps1
   ```
10. Observe:

    - The pipeline fails
    - Terraform does not run
    - A clear error message is displayed

```
OperationStopped: (Promotion block...lign versions).:Strir
Promotion blocked: prod (1.0.1) is ahead of test (1.0.0).
```

11. This demonstrates that the pipeline enforces rules — it does not assume intent.

Phase 5 – Promote Through the Pipeline

12. Promote network module v1.0.1 to TEST and then PROD using the pipeline.

Takeaways

- Pipelines enforce behaviour — they do not decide promotion
- Automation protects humans from mistakes
- Promotion remains intentional and auditable
- Terraform architecture did not change
- This model scales directly into CI/CD systems

In the next part of this lab, you will move this same promotion logic into a **real CI/CD system**, without changing the model you have just learned.

Part 2. Automated Promotion with Azure DevOps (ADO) Pipelines

Teaching Points

In Part 1 you performed promotion using a local pipeline. In this part of the lab, you take automating the **execution** of those same steps to using **Azure DevOps (ADO) Pipelines**, while keeping **promotion decisions and approvals visible and human-controlled**.

This lab remains **Pattern A**: parallel roots (DEV/TEST/PROD) with parallel state. You are **automating execution**, not changing environment strategy.

Network and security modules remain independent. In this lab you promote a new **network module version only**. The security module remains pinned.

The objective of this lab is to promote **network v1.0.2** through **DEV→TEST→PROD** using ADO Pipelines, with approvals.

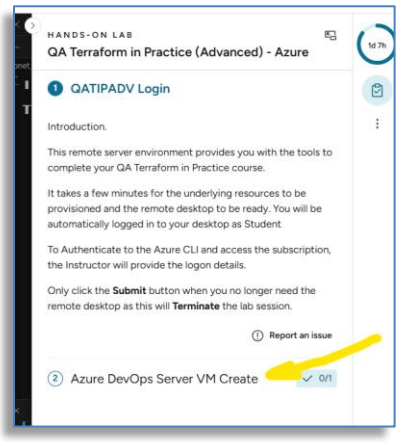
This part of the lab is organised into the following phases:

Phase 1 – Deploy the DevOps Server VM
Phase 2 – Create the ADO Project and Environment Roots Repository
Phase 3 – Deploy Self Hosted ADO Pipeline Agent
Phase 4 – Create Azure Authentication for Pipelines
Phase 5 – Configure ADO Environments and Approvals
Phase 6 – Add a Promotion Pipeline (Apply with approvals)
Phase 7 – Release Network v1.0.2 to dev
Phase 8 – Attempt an out of order Promotion

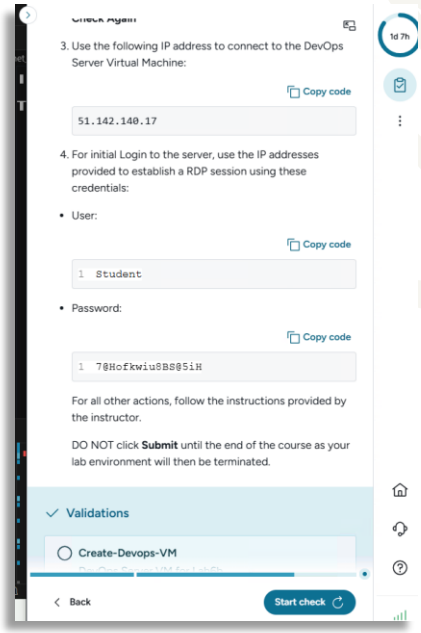
Phase 1 – Deploy the DevOps Server VM

Restrictions in the lab environment prevent the use of traditional web-based DevOps. An alternative is the use of a stand-alone DevOps Server which provides all of the functionality needed for controlled pipelining.

13. Expand the Lab sidebar and click on ‘Azure DevOps Server VM Create’…

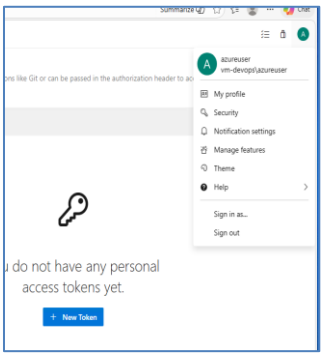


14. Scroll down and retrieve the IP address, username and password for the DevOps vm that will be created for you and then click on ‘Start check”…

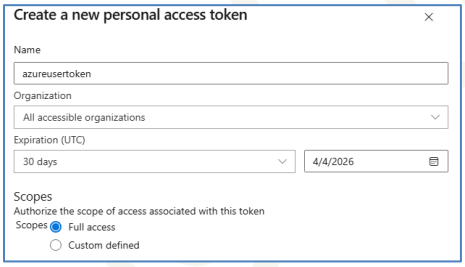


15. The deployment will progress and you will receive validation confirmation.
16. Open a web browser session to **https://qatipadv-ado/DefaultCollection**
17. You will receive a 'Your connection is not private' message

18. Click on 'Not secure' in the URL bar and select **Certificate details**
19. Select the Details tab and click on '**Export**'
20. Save **qatip-ado.crt** to Downloads
21. Navigate to Downloads and double-click **qatip-ado.crt**
22. Select Install Certificate > Local Machine > Place all certificates .... > Trusted Root Certificate Authorities > Next > Finish
23. Bookmarking existing links as needed, Close your existing browser session, open a new one and browse to **https://qatipadv-ado**
24. Log in using **azureuser/TF_P@ssw0rd_123**
25. Click your account Avatar > Security > New Token ..



26. Name the token '**usertoken**', select '**All accessible organizations**' and **grant Full Access** ..

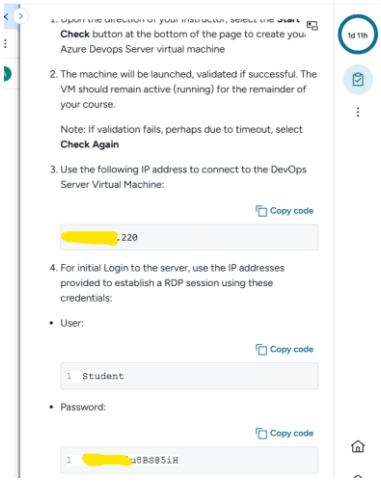


Note: In production environments, PAT scopes should be restricted according to the principle of least privilege.

27. Click **Create**
28. Copy the token and save it for later use.
29. Leave the browser session open as you will return to it shortly.

Install and configure the agent

30. In your lab vm, run **MSTC** to connect to the remote machine, using the IP address, username and password displayed on the lab sidebar.



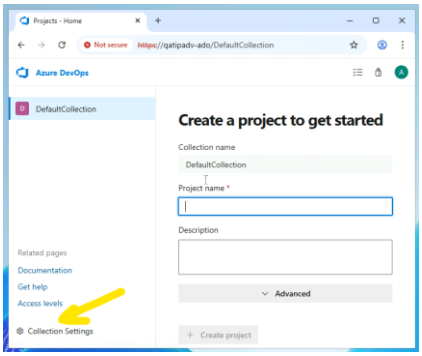
31. Right click the '**stage2-install-agent'** script on the devops vm desktop and select '**Run with Powershell**'
32. Provide your PAT key when prompted and allow the installation to proceed.
33. When the Powershell window closes, open a new command prompt and run...

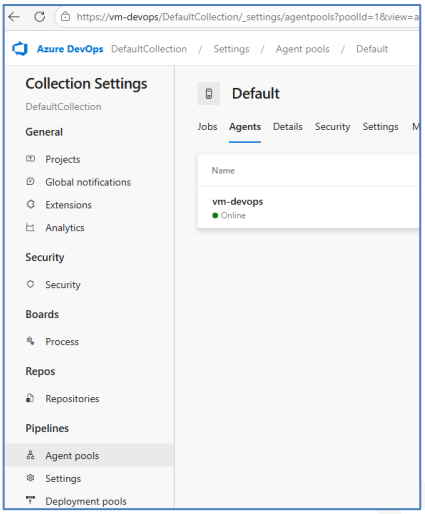
    c:\ado-agent\config.cmd

34. Configure as shown ....

    URL: **https://qatipadv-ado/DefaultCollection**
    Enter authentication type: **PAT**
    Agent Pool: **{enter}**
    Agent Name: **{enter}**
    Work folder: **{enter}**
    Run as Service: **Y**
    Enable unrestricted: **Y**
    User account: **{enter}**
    Prevent service start: **N**

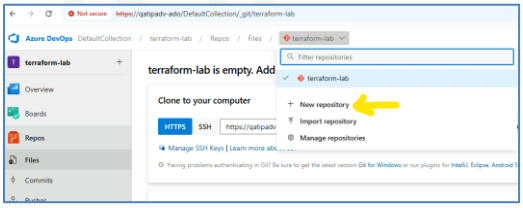
35. You can now sign out of the RDP session
36. In your student-vm, return to your qatipadv-ado browser session. If you have closed the session; **https://qatipadv-ado/DefaultCollection**, **azureuser/TF_P@ssw0rd_123**
37. Verify the agent is registered and online



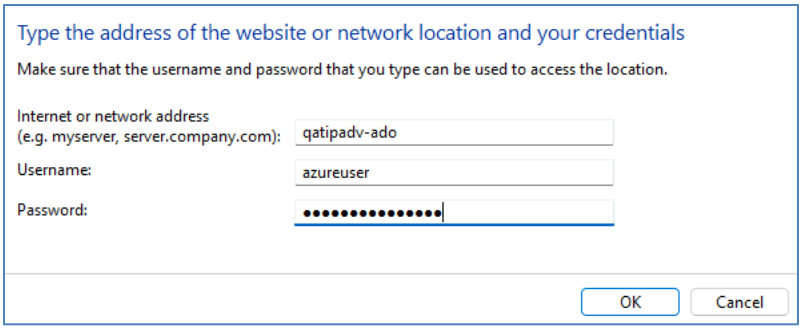


Phase 2 – Create the ADO Project Environment Roots Repository

38. Click on 'Azure DevOps' to return to the DefaultCollection landing page
39. Create a new project named '**terraform-lab**'
40. Inside the project, create a new Git repository: **env-roots**



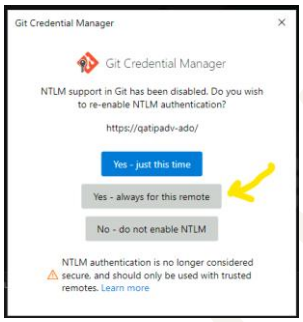
41. Delete the default '**terraform-lab'** repository
42. Open Credentials Manager in Control Panel and 'Add a Windows credential' as shown, using password **TF_P@ssw0rd_123** ...



43. Using VSC, clone the new empty repository locally …

```
cd\
mkdir c:\ado-repo
cd c:\ado-repo
git config --global http.sslBackend schannel
git clone https://qatipadv-ado/DefaultCollection/terraform-
lab/_git/env-roots
```

Enable NTLM support, always for this remote…



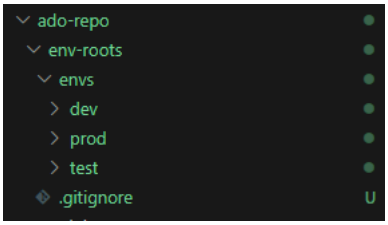

```
cd env-roots
git config --local credential.useHttpPath true
git config --local user.name  "azureuser"
git config --local user.email "azureuser@lab.local"
```

44. Add the repo folder to VSC Explorer; File > Add Folder to Workspace > c:\ado-repo
45. Delete **README.md** from your cloned ado repo
46. Copy your **lab6/guided/envs** folder into this repository.
47. Create a .gitignore with:

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
```

In this course, .tfvars files are committed intentionally.
They define environment configuration and are reviewed as part of promotion.

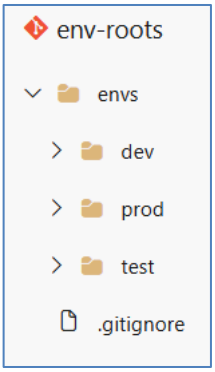
48. **Checkpoint**: Ensure your local env-roots is as below...



49. Commit and push:

```
cd c:\ado-repo\env-roots
git add .
git commit -m "Add environment roots (dev/test/prod)"
git push -u origin main
```

50. Switch to ADO and confirm the repo update...



Phase 3 – Configure Azure Authentication for Pipelines

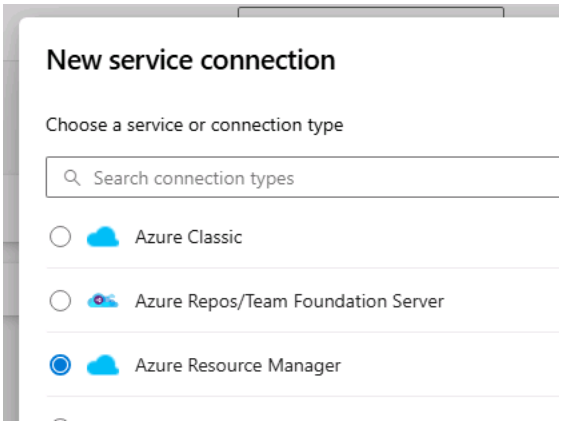
In this phase you create a **Service Connection** in DevOps, used so that pipelines can run Terraform, and you also identify the state remote backend.

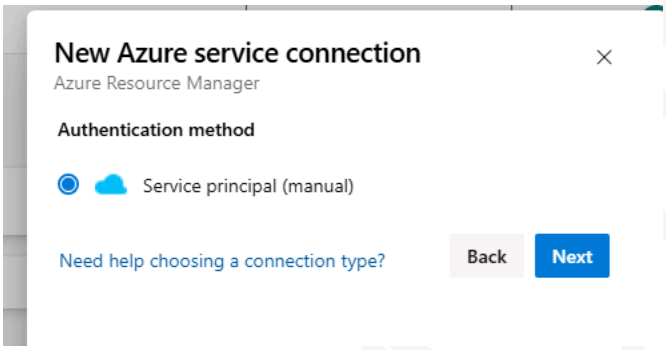
51. From VSC, Create a service principal, update with your subscription and a 4 random digit unique identifier

```
az ad sp create-for-rbac --name "ado-###" --role Contributor --
scopes /subscriptions/{subscription-id}
```

```
{
  "appId": "ee7c5602-d557-468d-bf00-c0605b943cbc",
  "displayName": "ado-1234",
  "password": "Y_f8Q~WtK_ml7My_MnUzzUTNRepG5yYMLyjcZakh",
  "tenant": "4cfe372a-37a4-44f8-91b2-5faf34253c62"
}
```

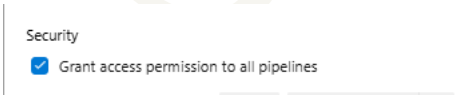
In DevOps, Project Settings, select Service connections ..





Populate with service principal details you have generated. Test connectivity and grant access to all pipelines ...

Environment: Azure Cloud
Scope Level: Subscription
Subscription Id: Your **subscription id**
Service principal id is the **appId**
Service principal key is the **password**
Name the service connection: **sc-terraform-lab**



* Ensure you grant access permissions to all pipelines.

Backend configuration

52. Retrieve your current Terraform remote backend values:

- storage_account_name
- container_name

- resource_group_name

53. Create a variable group (Pipelines > Library) …
54. Name: **vg-terraform-backend**
55. Variables:

    TF_BACKEND_SA : **advtfstate{your suffix}**
    TF_BACKEND_CONTAINER : **tfstate**
    TF_BACKEND_RG : **landing-zone-shared-tfstate-rg**
    ARM_SUBSCRIPTION_ID: **{your subscription id}**

56. Save changes

Phase 4 – Configure ADO Environments and Approvals

This phase enables **controlled promotion** using approvals.

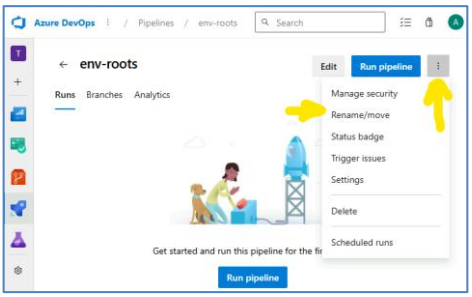
57. Create three environments (Pipelines > Environments) :
    - dev
    - test
    - prod
58. Add approvals (refresh screen if permission error initially displayed):
    - Leave **dev** without approval
    - Add an approver (azureuser) to **test**
    - Add an approver (azureuser) to **prod**
59. This mirrors real workflows: fast feedback in dev, deliberate control later.

Phase 5 – Add a Change Pipeline (Apply with approvals)

60. A pipeline that applies Terraform in DEV, then TEST, then PROD, enforcing execution order and approvals where needed, has been provided in the course-wide **artifacts**.
61. Create a folder **.azure-pipelines** under env-roots
62. Copy the supplied **change-all.yml** file:
    - From: artifacts/pipelines/change
    - To: env-roots/.azure-pipelines
63. From **ado-repo\env-roots**, commit and push to main …

```
cd c:\ado-repo\env-roots
git add .azure-pipelines/change-all.yml
git commit -m "Add change-all pipeline (apply with approvals)"
git push
```

64. In DevOps > Pipelines > Create Pipeline …
    - Azure Repos Git
    - Repository: **env-roots**
    - Existing YAML file
    - Branch: **main**
    - Path: .azure-pipelines/change-all.yml
65. **Save** the pipeline without running it. (drop-down to right of Run)
66. Rename the pipeline from '**env-roots**' to '**change-all**'



Phase 6 – Release Network v1.0.2 to DEV

Release v1.0.2 of the network module

67. In **module-repos/terraform-azure-module-network/locals.tf**, make a small non-breaking change …

    Update the manager tag from 'John Robinson (NetMod v1.0.1)' to '**Clare Hooper (NetMod v1.0.2)**'

68. Commit, Tag and Push the module change…

```
cd c:\module-repos\terraform-azure-module-network
git add locals.tf
git commit -m "Publish network module v1.0.2"
git tag v1.0.2
git push origin main
git push origin v1.0.2
```

Promote Network v1.0.2 to DEV

69. In **ado-repo/env-roots/envs/dev**, update **main.tf** …

    Change the **network** module source: ref=v1.0.1 → **ref=v1.0.2**

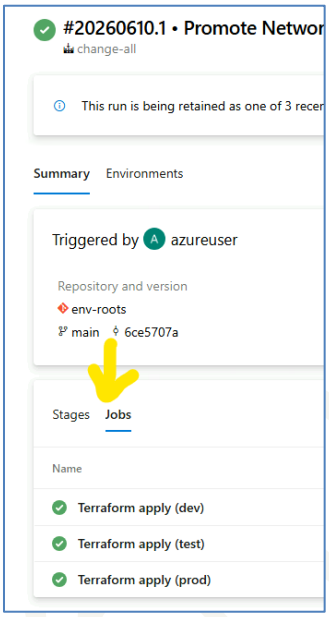
70. At **env-roots**, commit and push to main …

```
cd c:\ado-repo\env-roots
```

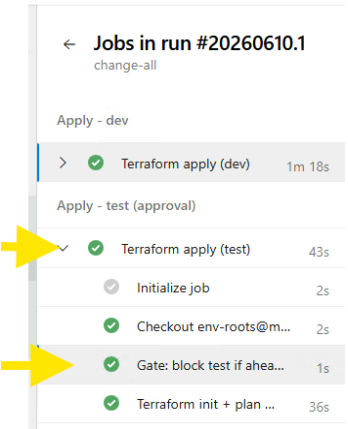
```
git add .\envs\dev\main.tf
git commit -m "Promote Network Module v1.0.2 to DEV"
git push
```

71. The change-all pipeline is triggered on updates to the main branch of the repo and therefore should now be in progress. In ADO, navigate to Pipelines>Recent and click into change-all...
    - Grant the permissions request (one-off for initial run of a pipeline)
    - Dev applies automatically without approval
    - Test waits for permissions and approval - Grant permission and Approve
    - Prod waits for permissions and approval - Grant permission and Approve

72. Observe the runs and verify the changes to DEV and no-ops to TEST and PROD...



Drill into the TEST run to verify no gate block, TEST is not ahead of DEV ....



```
Detected module versions (from main.tf):
  network  dev=1.0.2  test=1.0.1
  security dev=1.0.0  test=1.0.0
Gate passed: test is not ahead of dev for any governed module.
Finishing: Gate: block test if ahead of dev (all governed modules)
```

Drill into the PROD run to verify no gate block, PROD is not ahead of TEST

```
Detected module versions (from main.tf):
  network  test=1.0.1  prod=1.0.1
  security test=1.0.0  prod=1.0.0
Gate passed: prod is not ahead of test for any governed module.
Finishing: Gate: block prod if ahead of test (all governed modules)
```

Phase 7 – Attempt an out-of-order promotion (expected to fail)

Attempt to promote v1.0.2 to Prod

73. In env-roots, update **envs/prod/main.tf** …

    Change the network source: ref=v1.0.1 → **ref=v1.0.2**

74. At **env-roots**, commit and push to main …

```
cd c:\ado-repo\env-roots
git add .\envs\prod\main.tf
git commit -m "Promote Network Module v1.0.2 to PROD"
git push
```

75. The pipeline triggers:

    Dev applies automatically (no-op)

    Approve Test (no-op)

    Approve Prod (failure expected)

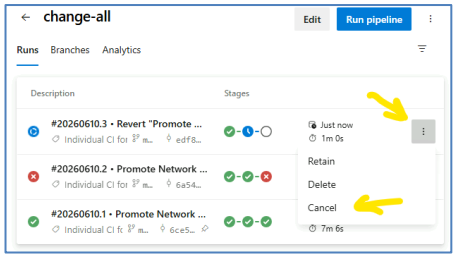
Drill into the PROD run to verify gate block, PROD is ahead of TEST

```
Detected module versions (from main.tf):
  network  test=1.0.1  prod=1.0.2
Promotion blocked: prod network (1.0.2) is ahead of test network (1.0.1). Promote in order.
```

76. Revert the incorrect promotion commit to main (restore prod to network v1.0.1)

```
cd c:\ado-repo\env-roots
git revert --no-edit HEAD
git push
```

77. This will trigger a no-op pipeline run. For expedience, cancel the run. (the run, do not delete the pipeline!)



78. In **env-roots**, update the environment configuration to reflect the correct promotion order:

    Update the **network** module to **v1.0.2** in **envs/test/main.tf** and **envs/prod/main.tf**

79. Commit and push to main …

```
cd c:\ado-repo\env-roots
git add .\envs\test\main.tf
git add .\envs\prod\main.tf
git commit -m "Promote Network Module v1.0.2 to TEST/PROD"
git push
```

80. Observe the promotion pipeline …

- Dev runs automatically (expected no-op)
- Approve Test (upgrade succeeds)
- Approve Prod (upgrade succeeds)

81. **Confirm**: Network module v1.0.2 has been promoted through DEV/TEST/PROD

82. **Confirm**: Security module v1.0.0 is still current through DEV/TEST/PROD

Key Takeaways

- ADO Pipelines automate **execution**, not decision-making
- Approvals keep promotion **intentional and auditable**
- Pattern A remains unchanged
- Modules are promoted independently
- Deterministic no-op applies build trust in automation

Where This Design Stops

We have intentionally automated **execution across environments** while leaving release intent **implicit**. Because a single central promotion pipeline is used:

- All environments run, even if only one changed
- Invalid promotion intent can exist briefly in Git
- Humans must correct mistakes (for example, reverting a pin)

These limitations are **intentional** at this stage of the course.

Part 3. Environment-Scoped Promotion Pipelines

Teaching Points

In Part 2 you automated Terraform execution across DEV, TEST, and PROD using a single promotion pipeline with approval gates. That design deliberately centralised execution so that ordering, approvals, and Terraform behaviour could be observed clearly.

However, it also made release intent implicit: the pipeline decided *where* to run, rather than *why* a promotion should occur.

That demonstrated *how* promotion can be executed safely and audibly, but it also deliberately exposed an important limitation: **automation alone does not enforce release intent**.

In this part, you evolve the design to a **production-grade promotion model** by introducing **one pipeline per environment**. Each environment advances **only when explicitly promoted**, preventing accidental unintentional leapfrogging and aligning automation with real-world release practices.

This part teaches the distinction between:

- execution automation
- release intent
- environment ownership

This part of the lab is organised into the following phases:

Phase 1 - Rethink Promotion Responsibility
Phase 2 - Add environment-scoped PR plan pipelines (dev/test/prod)
Phase 3 - Retire the change-all Pipeline
Phase 4 - Review and release Environment-Scoped Pipelines

Phase 1 - Rethink Promotion Responsibility

Before changing any pipelines, pause and reflect on the following principle:

Automation should never decide when to promote — only how to execute a promotion once the decision is made.

Previously:

- a single pipeline executed Terraform for all environments
- approvals gated execution
- but **release intent was implicit**

Moving forward:

- promotion intent becomes explicit
- each environment advances independently
- time separation between dev, test, and prod is restored

Phase 2 - Add environment-scoped PR plan pipelines (dev/test/prod)

A *plan pipeline* runs terraform plan automatically when a change is proposed, allowing humans to review what *would* change before anything is applied. It does not deploy resources or make decisions — it exists purely to make change visible early. These pipelines run **terraform init** and **terraform plan** against the target environment's state but never apply changes.

Earlier, plan pipelines were deliberately omitted because the focus was on understanding *execution behaviour*: how Terraform applies changes, how ordering and approvals work, and how environments converge safely. At that stage, adding PR-time validation would have distracted from the core lesson.

Now that environments are to advance independently, visibility must also become environment-specific. A change to DEV, TEST, or PROD represents a different level of risk and intent, and each requires its own review. Environment-scoped plan pipelines provide that visibility without changing how promotion itself works.

In this course we want the plan to be reviewed **before intent is declared**. We will therefore run Terraform plans automatically during the pull request (as an *optional* PR check) so reviewers can pause and consider impact while the change is still only a proposal or skip the planning if desired.

Merging the PR is the **explicit decision to apply the changes**. After merge, environment-scoped change pipelines run, and approvals act as the final control point before apply.

Objective

We want pull request validation to be environment-specific:

• Changes under envs/dev/** trigger a **dev** Terraform plan
• Changes under envs/test/** trigger a **test** Terraform plan
• Changes under envs/prod/** trigger a **prod** Terraform plan

The operator reviews the plan output and decides whether to merge.

Why environment-scoped PR plan pipelines?

Using one PR plan pipeline per environment keeps validation deterministic and easy to reason about.

A pull request touching dev should not run test or prod plans. There is no environment detection logic and no branching behaviour inside a pipeline.

This avoids brittle "detect which environment changed" scripting and keeps the model auditable, scalable, and aligned with the environment-scoped promotion pipelines introduced later in this lab.

Add the three env PR plan YAML files into the env-roots repo

In your local clone of **env-roots**:

1. Create a branch:

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D add-pr-plans
git checkout -b add-pr-plans
```

   ***note** branch -D is used here and in later code blocks in case of command re-runs

2. Copy these files from **artifacts/pipelines/plan** into **env-roots/.azure-pipelines/** ...

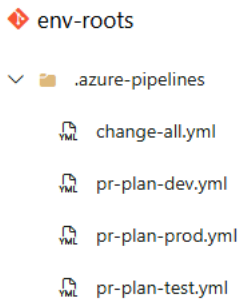
   pr-plan-dev.yml
   pr-plan-test.yml
   pr-plan-prod.yml

3. Commit + push:

```
cd c:\ado-repo\env-roots
git add .azure-pipelines/pr-plan-dev.yml
git add .azure-pipelines/pr-plan-test.yml
git add .azure-pipelines/pr-plan-prod.yml
git commit -m "Add PR Plan pipelines"
git push -u origin add-pr-plans
```

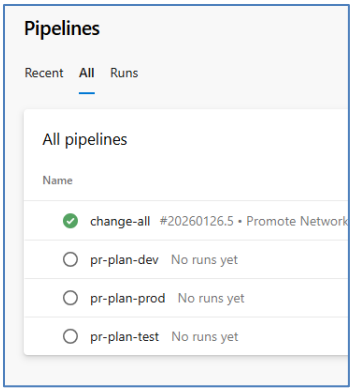
4. In DevOps > Repos > env-roots > Pull requests > **Create a pull request** to main and **Complete (merge)** it.

   Checkpoint: In ADO; Repos > env-roots > main > Files, you can see all three plan YAML files under **.azure-pipelines**/...



5. For each new YAML file, create a pipeline reference in Azure DevOps...
   - DevOps > Pipelines > New pipeline
   - Choose Azure Repos Git
   - Select repo: **env-roots**
   - Choose Existing Azure Pipelines YAML file
   - Branch: main
   - Path: select each of these in turn:
     - .azure-pipelines/pr-plan-dev.yml
     - .azure-pipelines/pr-plan-prod.yml
     - .azure-pipelines/pr-plan-test.yml
   - Rename the pipeline from env-roots to one of these as appropriate:
     - pr-plan-dev
     - pr-plan-prod
     - pr-plan-test

6. **Checkpoint:** You now have three new named pipelines listed under Pipelines...



Configure PR plan pipelines as optional checks

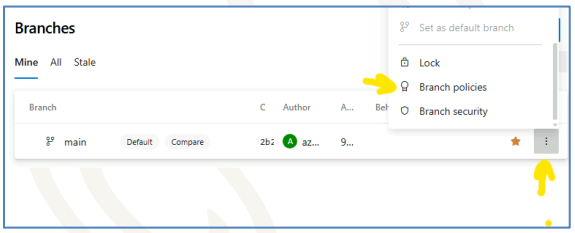
To ensure PR plan pipelines run automatically and are visible to reviewers, we attach them to the main branch as **optional build validation policies**.

These policies:

- make plan output visible during PR review
- do **not** block merges
- do **not** enforce governance yet

This preserves flexibility while making intent review possible.

7. DevOps > Repos > Branches - Locate branch: **main**
8. Select ⋯ More options→ Branch policies



9. Under Build validation, click Add

   Configure:

   Build pipeline: pr-plan-dev
   Path filter (optional): /envs/dev/**
   Trigger: Automatic
   Policy requirement: Optional

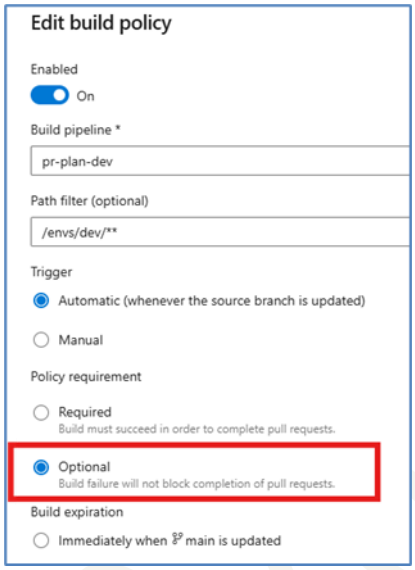
10. Click **Save**
11. Repeat these steps to create two more optional build validation policies:

Prod:

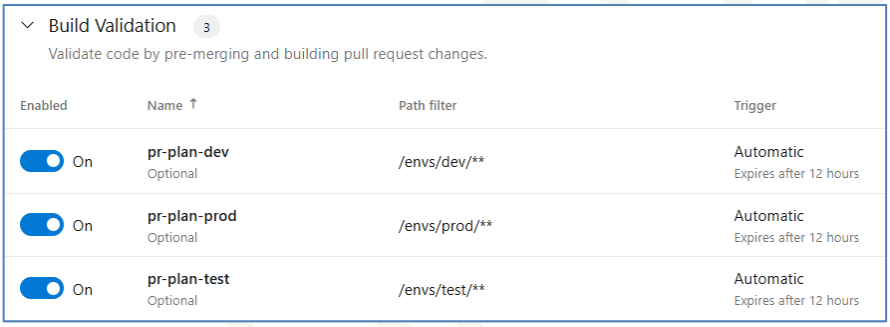
Build pipeline: pr-plan-prod
Path filter: /envs/prod/**
Trigger: Automatic
Policy requirement: Optional

Test:

Build Pipeline: pr-plan-test
Path filter: /envs/test/**
Trigger: Automatic
Policy requirement: Optional



Checkpoint: Verify validation policies are as shown...



Phase 3 - Retire the change-all Pipeline

12. Disable the single change pipeline, **change-all** (.azure-pipelines/promote.yml) to avoid duplicate runs once environment-scoped change pipelines are introduced.
13. In Azure DevOps, go to Pipelines → Pipelines → All → change-all
14. Edit the pipeline → ··· **edit**
15. Select ··· (More actions) → **settings**.
16. Under Processing of new run requests, select **Disabled**.
17. Click **Save**.

Phase 4 - Review and release Environment-Scoped Pipelines

18. Instead of one pipeline that runs all environments, you will now use pre-provisioned environment-scoped pipelines.

19. Review the following pipelines in the **artifacts\pipelines\change** folder ...

    - change-dev.yml
    - change-test.yml
    - change-prod.yml

20. Each pipeline:

    - targets one environment only
    - runs Terraform only for that environment
    - is triggered only when that environment's configuration changes
    - has a feature selection parameter, used to vary functionality, currently set at 1

21. Copy the supplied YAML from the lab artifacts into your local env-roots ...

    From: **artifacts\pipelines\change**

    change-dev.yml
    change-prod.yml
    change-test.yml

    To: **env-roots\.azure-pipelines**

22. At **env-roots**; Create Branch, Commit and push ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D add-env-change-pipelines
git checkout -b add-env-change-pipelines
git add .azure-pipelines/change-*.yml
git commit -m "Add env promotion pipelines"
git push -u origin add-env-change-pipelines
```

23. In DevOps > Repos > env-roots > Pull requests > Create a pull request to main and Complete (merge) it.

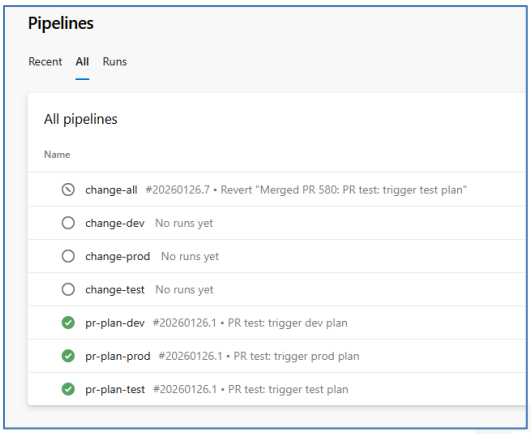
24. In Azure DevOps > Pipelines, create and save (without running), pipeline references for each YAML file.

    change-dev.yml
    change-prod.yml

change-test.yml

25. Rename each pipeline to:

    change-dev
    change-prod
    change-test



Important: Double-check that you have selected the ***change*** yaml files, not ***plan***.

Review the following status:

- Each environment change pipeline runs **only when its config changes**
- Dev can advance independently
- Test and prod require explicit change approval
- Approval gates are meaningful
- No environment can "jump ahead" *accidentally*, it must be a deliberate action

End-to-End Perspective (Read Before Moving On)

You have now seen all the components involved in a controlled change flow, even though they were introduced in stages. When viewed end-to-end, a typical change follows this sequence:

*The following describes the conceptual flow you have built, **not** a new set of steps to perform now.*

1. An environment change is made, the adoption of a new module version for example.
2. A pull request is opened proposing that environment change.
3. An **environment-scoped PR plan pipeline** runs, showing what would change if the proposal were accepted.

4. The plan is reviewed while the change is still only a proposal.
5. The pull request is merged - this is the moment the change is accepted.
6. An **environment-scoped change pipeline** runs for that environment.
7. An approval is requested where required (test and prod).
8. Terraform applies the change.

Each environment follows this same flow independently. Nothing is changed automatically, and nothing runs without a deliberate action somewhere in the chain.

A full walkthrough of this sequence — from module release through to applied promotion — will take place shortly.

Reflection: Something Still Is not Explicit

As at now, all environments are updated selectively and then pipelines run with approval requests where appropriate. Nothing happens accidentally.

And yet, something important is still missing. Pause for a moment and consider this question carefully:

Where did the decision to change this environment actually happen?

Not where it was *executed*. Not where it was *approved*. Not where Terraform *applied* changes. Where did someone explicitly say: "We are changing this environment now."

You may find that the answer is unclear. The change may have been inferred from: a version change, a pipeline trigger, an approval click.

But inference is different from declaration. At small scale, this ambiguity is tolerable. At larger scale, it becomes dangerous. We still need to address one missing requirement:

How do we make change intent explicit — and ensure automation cannot act without it?

Do not try to solve this yet. Simply notice the gap.

Part 4. Forensic Review of a Production Promotion

Teaching Points

You will now promote a **new module version** to **production** using the promotion architecture you have already built.

Nothing in this lab is experimental.
Nothing in this lab is unsafe.
Nothing in this lab requires new tooling.

That is intentional.

The purpose of this run is not to *fix* the change process, but to **observe it carefully** and ask the hard question just posed …

After a real production change, can we point to where the decision to change configuration was explicitly recorded?

Phase 1 – Create a New Module Version

In previous labs you designed and assembled environment change architecture, but you have not yet stepped through a complete production change from start to finish.

In this lab, you will deliberately exercise that complete architecture for the first time and observe how a real production change might unfold.

Release a new Security module version

In the **terraform-azure-module-security module repository**:

1. Open the existing **locals.tf**
2. Update the **manager value** from 'Michael Coulling-Green (SecMod v1.0.0)' to '**Peter Smith (SecMod v1.0.1)**'

This is a deliberately boring change. The change itself is not the point.

Commit and tag

From the **terraform-azure-module-security** module repository:

```
cd c:\module-repos\terraform-azure-module-security
git add locals.tf
git commit -m "Update manager"
git push origin main
git tag v1.0.1
git push origin v1.0.1
```

Confirm that the new tag exists remotely.

At this point, you have **published a new module version** — nothing more.

3. Take a moment to consider:
    - Have you stated *anywhere* that v1.0.1 is approved for production?

- Or have you simply created a version that *could* be used?

Phase 2 – Promote new security module to PROD

Promotion in this architecture happens by **pinning a module version in an environment root**.

Create a branch in env-roots

4. In the env-roots repository:

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D change-prod-1
git checkout -b change-prod-1
```

This branch exists solely to propose a production change.

Update the PROD module reference

5. Open: envs/prod/**main.tf**
6. Locate the **security** module source and update the version reference:

?ref=v1.0.0 > **?ref=v1.0.1**

Do **not** change dev or test.

This is a **deliberate production change**, not an accidental leapfrog.

7. Ask yourself:

- Does changing ?ref= ***declare*** a promotion decision?
- Or does it **create** a configuration change that ***results*** *in* promotion later?

Both answers may feel partially true — note that tension.

Commit and push

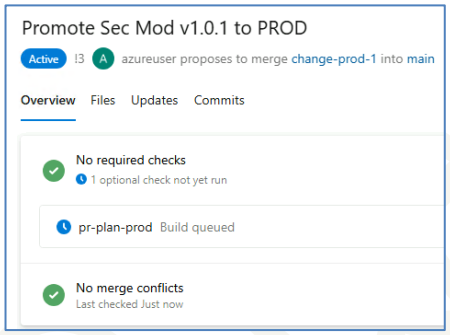
```
git add envs/prod/main.tf
git commit -m "Promote Sec Mod v1.0.1 to PROD"
git push -u origin change-prod-1
```

Phase 3 – Pull Request and Validation

Create the Pull Request

8. In Azure DevOps:
    - Create a PR targeting main
    - Review the diff carefully by looking at the changed files…

    - Observe the PR validation pipeline (pr-plan-prod) running, granting permission as needed…

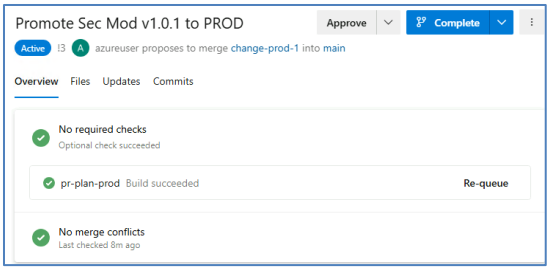


9. Look at the PR and ask:
    - If someone read this PR in six months' time, would they clearly see:

      "This is a production promotion decision"?
    - Or would they merely infer that meaning from context?

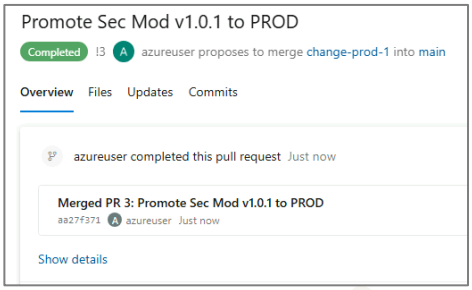
Merge the PR

10. Once the optional pr-plan-prod pipeline has completed successfully, complete the PR..



11. Record:

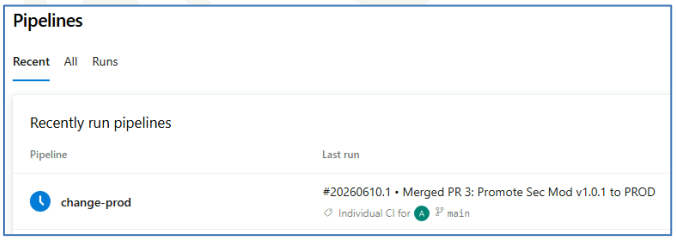
- PR number
- Author
- Merge date/time
- Secure Hash Algorithm (SHA)



Phase 4 – Observe the Production Pipeline

Observe pipeline execution

12. Navigate to: Pipelines → Pipelines
13. Open the **change-prod** run triggered by the **merge to main**...



14. Confirm:

- Only the production pipeline runs
- The prod environment is targeted
- The correct module version is applied

Mechanically, everything should look correct.

15. Ask yourself:

- Where does Azure DevOps explicitly state that this run represents a **promotion decision**?
- Or are you inferring that from file paths, YAML names, and context?

Phase 5 – Approval and Execution

Approve the production gate

16. Grant the permissions needed as this is the first run of this pipeline.
17. When the pipeline pauses:
    - Review the approval request
    - Approve (or observe approval)
18. Record:
    - Who approved
    - When
19. Consider carefully:
    - Does this approval say:

      "We choose to promote production to vX.Y.Z"?
    - Or does it say:

      "This pipeline is allowed to continue"?

      That distinction matters.

Observe Terraform apply

20. Allow the pipeline to complete.

    Confirm:

    - Apply succeeds
    - The security manager value is updated to **Peter Smith** in production

    Everything has worked exactly as designed.

Phase 6 – Forensic Review

Gather your evidence

21. Collect:

- Module version referenced in production configuration

Where to find it:
In the production environment configuration:

- envs/prod/main.tf
- The module source reference (for example ref = "v1.0.1")

This change will also be visible in the production pull request diff.

Why it matters:
This shows what version of the module production is configured to use.
It represents the *technical content* of the promotion — but not the decision itself.

On its own, this is just configuration. It does not tell you when or why production adopted this version.

- Production pull request

  Where to find it:
  In Azure DevOps...

  - Repos → Pull Requests → Completed
  - Locate the PR that modified envs/prod/main.tf

  Why it matters:
  The production pull request represents the formal proposal to change production configuration.

  It shows:

  - who proposed the change
  - what was changed
  - when the change was accepted

  However, the PR does not confirm that production was actually updated — only that the change was approved to be merged.

- Promotion pipeline run

  Where to find it:
  In Azure DevOps...

  - Pipelines → Runs
  - Locate the run of the production promotion pipeline triggered by the PR merge

  The run will typically reference the commit SHA from the merged PR.

  Why it matters:
  This pipeline run represents execution.
  It is the point where automation acted on the accepted change.

  This answers the question:

“When did the platform attempt to promote production?”

But it still does not tell you why that promotion was authorised.

- Approval record

  Where to find it:
  In Azure DevOps …

  - Environments → prod → History
  - Select the relevant pipeline run and view its approvals

  Why it matters:
  The approval record shows who allowed execution to proceed and when.

  This is the strongest human signal in the system — but it is still procedural, not declarative. An approval allows execution; it does not state *intent* in a durable, auditable form.

- Commit SHA

  Where to find it:

  - In the merged production pull request, or
  - In the promotion pipeline run details

  It will look like a short hexadecimal string (for example dade5c30).

  Why it matters:
  The commit SHA is the immutable identifier for this promotion event.

  It cryptographically ties together:

  - the production PR
  - the pipeline run
  - the approval
  - the resulting state change

  It proves *what happened*, but not *why it happened*.

You now have a full promotion trail.

Look for explicit intent

22. Try to find **one artifact** that explicitly states:

    “We decided to promote production to vX.Y.Z.”

    Do not infer. Do not combine clues. Look for a single, clear declaration.

Expected Outcome

Your conclusion should be: The promotion was deliberate, controlled, approved, and successful. However, the decision to promote exists only as inferred behaviour, not as an explicit artifact.

Why This Matters

This lab proves something subtle but important:

- **Control** is not the same as **clarity**
- **Deliberate action** is not the same as **declared intent**
- **Successful automation** can still fail an audit question

Transition to next lab

In the next lab, you will redesign this flow so that:

- Changes are categorized
- Change intent is declared **before** execution
- Pipelines refuse to run without it
- Promotion decisions are explicit, durable, and auditable

For now, resist the urge to fix anything. You have just discovered *why* a fix is needed.

***** Congratulations, you have completed this lab *****

Copyright